Security Vulnerability Report

Executive Summary

Scan Overview:
• Scan ID: dafba102-c39f-4583-a6fc-45594933569c
• Date: 2025-11-09T06:47:38.399Z
• Files Scanned: 1
• Total Vulnerabilities: 14

Severity Breakdown:
• Critical: 0
• High: 4
• Medium: 8
• Low: 2

Vulnerability Summary

File	Line	Type	Severity	CWE
files-1762670858408-844693053.py	38	subprocess_popen_with_shell_equals_true	High	CWE-78
files-1762670858408-844693053.py	81	hashlib	High	CWE-327
files-1762670858408-844693053.py	25	sqlalchemy-execute-raw-query	High	CWE-89: Improper Neutralization of Special Elements used in an SQ
files-1762670858408-844693053.py	38	subprocess_popen_with_shell_equals_true	High	CWE-78: Improper Neutralization of Special Elements used in an OS Com
files-1762670858408-844693053.py	24	hardcoded_sql_expressions	Medium	CWE-89
files-1762670858408-844693053.py	46	blacklist	Medium	CWE-78
files-1762670858408-844693053.py	55	blacklist	Medium	CWE-502
files-1762670858408-844693053.py	63	hardcoded_tmp_directory	Medium	CWE-377
files-1762670858408-844693053.py	25	formatted-sql-query	Medium	CWE-89: Improper Neutralization of Special Elements used in an SQ
files-1762670858408-844693053.py	46	eval-detected	Medium	CWE-95: Improper Neutralization of Directives in Dynamically Evalu
files-1762670858408-844693053.py	55	avoid-pickle	Medium	CWE-502: Deserialization of Untrusted Da
files-1762670858408-844693053.py	81	md5-used-as-password	Medium	CWE-327: Use of a Broken or Risky Cryptographic
files-1762670858408-844693053.py	31	blacklist	Low	CWE-78
files-1762670858408-844693053.py	49	blacklist	Low	CWE-502

Detailed Vulnerability Findings

Finding 1: subprocess_popen_with_shell_equals_true

File: files-1762670858408-844693053.py
Line: 38
Severity: High
CWE ID: CWE-78
Detection Tool: Bandit
Confidence: 90%

Description:
subprocess call with shell=True identified, security issue. Recommendation: Review the code for security implications and apply secure coding practices.

Finding 2: hashlib

File: files-1762670858408-844693053.py
Line: 81
Severity: High
CWE ID: CWE-327
Detection Tool: Bandit
Confidence: 90%

Description:
Use of weak MD5 hash for security. Consider usedforsecurity=False Recommendation: Use secure hash functions. Prefer SHA-256 or better.

Finding 3: sqlalchemy-execute-raw-query

File: files-1762670858408-844693053.py
Line: 25
Severity: High
CWE ID: CWE-CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option. Recommendation: Use parameterized queries (prepared statements) instead of string concatenation. Example: ```python cursor.execute("SELECT * FROM users WHERE id = %s", [user_id]) ```

Finding 4: subprocess-shell-true

File: files-1762670858408-844693053.py
Line: 38
Severity: High
CWE ID: CWE-CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Found 'subprocess' function 'call' with 'shell=True'. This is dangerous because this call will spawn the command using a shell process. Doing so propagates current shell settings and variables, which makes it much easier for a malicious actor to execute commands. Use 'shell=False' instead. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

Finding 5: hardcoded_sql_expressions

File: files-1762670858408-844693053.py
Line: 24
Severity: Medium
CWE ID: CWE-89
Detection Tool: Bandit
Confidence: 50%

Description:
Possible SQL injection vector through string-based query construction. Recommendation: Use parameterized queries or ORM methods instead of string formatting for SQL queries.

Finding 6: blacklist

File: files-1762670858408-844693053.py
Line: 46
Severity: Medium
CWE ID: CWE-78
Detection Tool: Bandit
Confidence: 90%

Description:
Use of possibly insecure function - consider using safer ast.literal_eval. Recommendation: Avoid using eval() as it can execute arbitrary code. Use ast.literal_eval() for safe evaluation.

Finding 7: blacklist

File: files-1762670858408-844693053.py
Line: 55
Severity: Medium
CWE ID: CWE-502
Detection Tool: Bandit
Confidence: 90%

Description:
Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible security issue. Recommendation: Avoid using pickle for deserialization. Use safer alternatives like JSON.

Finding 8: hardcoded_tmp_directory

File: files-1762670858408-844693053.py
Line: 63
Severity: Medium
CWE ID: CWE-377
Detection Tool: Bandit
Confidence: 70%

Description:
Probable insecure usage of temp file/directory. Recommendation: Use secure temporary file creation methods like tempfile.mkstemp() instead of hardcoding paths.

Finding 9: formatted-sql-query

File: files-1762670858408-844693053.py
Line: 25
Severity: Medium
CWE ID: CWE-CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Detected possible formatted SQL query. Use parameterized queries instead. Recommendation: Use parameterized queries (prepared statements) instead of string concatenation. Example: ```python cursor.execute("SELECT * FROM users WHERE id = %s", [user_id]) ```

Finding 10: eval-detected

File: files-1762670858408-844693053.py
Line: 46
Severity: Medium
CWE ID: CWE-CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
Detection Tool: Semgrep
Confidence: 85%

Description:
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this content can be input from outside the program, this may be a code injection vulnerability. Ensure evaluated content is not definable by external sources. Recommendation: Review the code for security implications and consider using secure coding practices appropriate for your use case.

AI-Generated Recommendations

• Implement parameterized queries for all database operations to prevent SQL injection
• Use environment variables for storing sensitive configuration data
• Implement proper input validation and output encoding
• Use strong cryptographic functions (bcrypt, scrypt, or Argon2) for password hashing
• Regular security code reviews and automated scanning
• Keep dependencies updated and monitor for known vulnerabilities